

Gnn

September 1, 2026

Contents

1	Meta (install extra packages)	2
2	Imports	2
2.1	Personal Infection Library	2
2.2	Generic Imports	2
2.3	Neural Network Libraries	2
3	Dataset Creation Functions	3
3.1	Custom Graph Constraints	3
3.2	Our individual functions to obtain relevant vectors	4
3.3	Constructing Single Data	5
3.4	Single Dataset creation example	6
3.5	Full Dataset	6
4	Model Construction	7
4.1	Model 1: GCN Layers	8
4.2	Model 2: GINE Convolutional Layers for edge feature message passing	8
4.3	Model 3: Softmax Model with GCNConv	9
4.4	Model 4: Graph Attention Network	9
5	Training	10
5.1	Dataset Partition	10
5.2	Dataset criteria	10
5.3	Training Routine	12
5.4	Testing Function	13
5.5	Model Routine	14

1 Meta (install extra packages)

```
source ../.venv/bin/activate
pip install poetry
pip install torch_geometric
```

```
cd ../infection_propagation
poetry install
cd ../neural-net-research
```

```
pwd
```

```
#!/../.venv/bin/python
```

2 Imports

2.1 Personal Infection Library

```
import importlib
from infection_propagation import Graph
from infection_propagation import Figures
from infection_propagation import Generators
```

```
importlib.reload(Graph)
importlib.reload(Figures)
importlib.reload(Generators)
```

2.2 Generic Imports

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as dists
import scipy.spatial as sp
import networkx as nx
import pandas as pd

from itertools import product, combinations
from collections import defaultdict
import random
import json
import time
import random
```

2.3 Neural Network Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import TensorDataset # DataLoader, TensorDataset
```

```

import torch_geometric.nn as gnn
import torch_geometric.data as gnn_data
import torch_geometric.utils as gnn_utils
from torch_geometric.loader import DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

```

3 Dataset Creation Functions

3.1 Custom Graph Constraints

We have certain constraints we'd like on observers for now for our different types of graphs.

For example, we'd like to break symmetry and/or "fix" the possible lengths the long arc can take in the circular graph. So we define the following constraint which we can optionally specify in our call below

```

def fix_arc(graph, observers, arc=0.56):
    length = graph.n
    arc_min = int((arc-0.05)*length)
    arc_max = int((arc+0.05)*length)
    arc_len = int(random.uniform(arc_min, arc_max))
    while True:
        new_arc = abs(observers[0] - observers[1])
        if new_arc != arc_len and new_arc != (length - arc_len):
            observers = [observers[0], (observers[0] + arc_len)%length]
            return observers
        else:
            return observers

```

Optionally, here's the constraint to merely break symmetry

- Note we hard code to sample 2 nodes since it MUST be the case that we only have the observer symmetry risk if we have two nodes.

```

def break_symmetry(graph, observers):
    length = self.n
    while True:
        if abs(observers[0] - observers[1]) == length / 2:
            observers = graph.sample_nodes(2)
        else:
            break
    return observers

```

For our line graph we absolutely require our observers to be end points so we'll utilize the below constraint function.

```

def force_end_points(graph, observers):
    vertex_end_points = [0, graph.n - 1]
    return vertex_end_points

```

Again we can alternatively define a constraint to force single end point nodes.

```
def force_end_point(graph, observers):
    vertex_end_points = [0, graph.n - 1]
    observers = [random.choice(vertex_end_points)]
    return observers
```

3.2 Our individual functions to obtain relevant vectors

NOTE: extra_features for our edge features is hard coded for IID $X \sim \text{Exp}(1)$

```
def get_observer_vector(times, observers, n):
    t = defaultdict(int)
    times = np.array([times]).flatten()
    observers = np.array([observers]).flatten()
    for time, node in zip(times, observers):
        t[node] = time
    return [[t[node]] for node in range(n)]

def get_observer_parent(times, observers, parents, n):
    t = defaultdict(lambda: [-1])
    times = np.array([times]).flatten()
    observers = np.array([observers]).flatten()
    for time, node in zip(times, observers):
        node_key = node
        t[node] = [parents[node_key]]
    return [t[node] for node in range(n)]

# Expecting Single SRC
def get_line_expected_infection_times(n):
    exp_time = lambda node, src: abs(node-src)
    infection_times_feature = []
    for src in range(n):
        expected_times = [
            exp_time(node, src) for node in range(n)
        ]
        infection_times_feature.append(expected_times)
    return infection_times_feature

def get_circle_expected_infection_times(n):
    exp_time = lambda x: -abs(x - n/2) + n/2
    infection_times_feature = []
    for src in range(n):
        expected_times = [
            exp_time(src - node) for node in range(n)
        ]
        infection_times_feature.append(expected_times)
    return infection_times_feature

def get_source_vector(test_src, n):
    return [(node == test_src) for node in range(n)]

def get_edge_features(graph, extra_features=False):
    graph.graph.sim_all()
```

```

gdf = graph.graph._adjacency_matrix.stack()
edges = gdf[gdf != np.inf].index
features = gdf[gdf != np.inf]
edge_list = [], []
feature_list = []

for u, v in edges:
    edge_list[0].append(u)
    edge_list[1].append(v)
    feature = [features[(u,v)]]
    if extra_features:
        x = np.linspace(0,2,10)
        y = dists.expon.pdf(x)
        feature = np.array(feature)
        feature = np.concat((feature, y))

    feature_list.append(feature)

edge_list = np.array(edge_list)
feature_list = np.array(feature_list)

return edge_list, feature_list

```

This is a unit of data in the training set.

- Note that `data.edge_index` has `dtype=torch.long` because it doesn't get "mutated by the network" is my best understanding

```

def create_datum_object(x, y, edge_list, edge_feature_list):
    data = gnn_data.Data()
    data.x = x
    data.y = torch.tensor(y, dtype=torch.float32)
    data.edge_index = torch.tensor(edge_list, dtype=torch.long)
    data.edge_attr = torch.tensor(edge_feature_list, dtype=torch.float32)
    return data

```

3.3 Constructing Single Data

Note: known named arguments go after `*args`

- make data, using `extra_features` or not
- `extra_features` includes observer infection time and infecting node
 - `infectiontime`: t or 0
 - `infectingnode`: `parents[node]` or `-1` (since 0 can be a node for Circle and Line graphs)

```

def make_data(
    srcs, dsts, graph_class, *args,
    extra_edge_features=False, parent_node_feature=False,
    extra_node_features=False, **kwargs
):
    g = graph_class(*args, **kwargs)

```

```

n = len(g.graph.vertices())
test_src, observers = g.get_source_observer_pairs(srcs, dsts, **kwargs)
times = g.simulation_trial(test_src, observers, iters=1, fixed_graph=True)
x = get_observer_vector(times, observers, n)
x = torch.tensor(x, dtype=torch.float32)
if parent_node_feature:
    xp = torch.tensor(
        get_observer_parent(times, observers, g.graph._parent, n),
        dtype=torch.float32
    )
    x = torch.cat((x, xp), 1)
if extra_node_features:
    if graph_class == Generators.LineIIDExpGraph:
        xp = torch.tensor(
            get_line_expected_infection_times(n),
            dtype=torch.float32
        )
    else: # Generators.CircleIIDExpGraph
        xp = torch.tensor(
            get_circle_expected_infection_times(n),
            dtype=torch.float32
        )
    x = torch.cat((x, xp), 1)
y = get_source_vector(test_src, n)
edge_list, edge_feature_list = get_edge_features(g, extra_features=
    extra_edge_features)
return create_datum_object(x, y, edge_list, edge_feature_list)

```

3.4 Single Dataset creation example

```

graph_type = Generators.CircleIIDExpGraph
# graph_type = Generators.LineIIDExpGraph
# observer_constraint = fix_arc
# observer_constraint = force_end_points
x = make_data(1, 2, graph_type, 20, extra_features=False, observer_constraints=
    observer_constraint)
x.x

```

3.5 Full Dataset

Parameters we set

- Graph Size and topology (could vary later on)
- Dataset with or without extra features
- Additional observer constraints
- Dataset size

```

dataset = []
observers = 2 # or 1
# graph_type = Generators.CircleIIDExpGraph

```

```

graph_type = Generators.LineIIDExpGraph
# observer_constraint = fix_arc
observer_constraint = force_end_points
# more_features=False
graph_size = 30
iters = 100

end_points = True

for _ in range(iters):
    n = random.randint(20, graph_size)
    x = make_data(
        1, observers, graph_type, n,
        extra_edge_features=True,
        parent_node_feature=False,
        extra_node_features=False,
        observer_constraints=observer_constraint
    )

    dataset.append(x)

dataset[0].y.shape[1]

```

4 Model Construction

Our activation function was previously a sigmoid but was swapped out for tanh due to its convergence improvements over sigmoid

- it's less of a victim of the vanishing gradient problem
- it's output is $[-1, 1]$ so still a sort of "probability" interpretation
- tanh seems to result in wilder final accuracies

Future Improvements:

- Maybe use a dropout layer?

Model Architecture:

1. Layer 1: Graph Convolution
 - Activation: tanh, extract feature probabilities
2. Layer 2: Graph Convolution
 - Activation: tanh, extract feature probabilities
3. Layer 3: Graph Convolution
 - Leave Evaluation to External Sigmoid Call

4.1 Model 1: GCN Layers

Traditional Graph Convolutional Network

```
num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        torch.manual_seed(1234)
        self.conv1 = gnn.GCNConv(num_features, 16)
        self.conv2 = gnn.GCNConv(16, 16)
        self.conv3 = gnn.GCNConv(16, num_predictions)

    def forward(self, x, edge_index, edge_weight):
        x = self.conv1(x, edge_index, edge_weight)
        x = x.relu()
        # x = F.dropout(x, p=0.5, training=self.training)
        x = self.conv2(x, edge_index, edge_weight)
        x = x.relu()
        # x = F.dropout(x, p=0.5, training=self.training)
        x = self.conv3(x, edge_index, edge_weight)
        return x

model = GCN()
pos_weight = torch.tensor([20/1])
criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
y_post_processing = False
```

None

4.2 Model 2: GINE Convolutional Layers for edge feature message passing

Considering more edge features than just the edge weight so a randomized weighted laplacian is not enough and a more complex model is needed following GINE

```
num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
edge_dim = dataset[0].edge_attr.shape[1]
class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        torch.manual_seed(1234)
        # self.conv1 = gnn.GINEConv(num_features, 16)
        # self.conv2 = gnn.GINEConv(16, 16)
        # self.conv3 = gnn.GINEConv(16, num_predictions)
        self.conv1 = gnn.GINEConv(nn.Sequential(nn.Linear(num_features, 16), nn.ReLU(), nn
            .Linear(16, 16)), edge_dim=edge_dim)
        self.conv2 = gnn.GINEConv(nn.Sequential(nn.Linear(16, 16), nn.ReLU(), nn.Linear
            (16, 16)), edge_dim=edge_dim)
        self.conv3 = gnn.GINEConv(nn.Sequential(nn.Linear(16, 16), nn.ReLU(), nn.Linear
            (16, num_predictions)), edge_dim=edge_dim)

    def forward(self, x, edge_index, edge_weight):
        x = self.conv1(x, edge_index, edge_weight)
        x = x.relu()
        x = self.conv2(x, edge_index, edge_weight)
```



```

        x = x.relu()
        x = self.conv3(x, edge_index, edge_weight)
        return x

model = GCN()
pos_weight = torch.tensor([20/1])
criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
y_post_processing = False

```

4.3 Model 3: Softmax Model with GCNConv

Here we hard focus on our true goal of predicting only one source by adding softmax in our post-training outputs

```

num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
class softGCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        torch.manual_seed(1234)
        self.conv1 = gnn.GCNConv(num_features, 16)
        self.conv2 = gnn.GCNConv(16, 16)
        self.conv3 = gnn.GCNConv(16, num_predictions)

    def forward(self, x, edge_index, edge_weight):
        x = self.conv1(x, edge_index, edge_weight)
        x = x.relu()
        x = self.conv2(x, edge_index, edge_weight)
        x = x.relu()
        x = self.conv3(x, edge_index, edge_weight)
        # x = gnn_utils.softmax(x, dim=0)
        return x

model = softGCN()
pos_weight = torch.tensor([20/1])
criterion = torch.nn.CrossEntropyLoss()
y_post_processing = True

```

4.4 Model 4: Graph Attention Network

Experimenting with alternative node message passing models

```

num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
class GAT(torch.nn.Module):
    def __init__(self):
        super().__init__()
        torch.manual_seed(1234)
        self.conv1 = gnn.GATConv(num_features, 16)
        self.conv2 = gnn.GATConv(16, 16)
        self.conv3 = gnn.GATConv(16, num_predictions)

    def forward(self, x, edge_index, edge_weight):
        x = self.conv1(x, edge_index, edge_weight)
        x = x.tanh()
        x = self.conv2(x, edge_index, edge_weight)

```

```

        x = x.tanh()
        x = self.conv3(x, edge_index, edge_weight)
        return x

model = GAT()
pos_weight = torch.tensor([20/1])
criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
y_post_processing = False

```

5 Training

5.1 Dataset Partition

80/20 or 90/10 dataset training partition?

```

train_data, test_data = train_test_split(dataset, test_size=0.2, random_state=42)
train_data = DataLoader(train_data, batch_size=10)
test_data = DataLoader(test_data)

```

5.2 Dataset criteria

Code Notes:

- `lr` is learning rate, how steep descent is
- `.BCEWithLogitsLoss` is the loss function
- `pos_weight = len(y)/sum(y)` is hard coded rn, adds weight to our source guess
- `lr_scheduler.ReduceOnPlateau` automatically updates our learning rate by a factor of around 10^{-1}
 - This occurs if loss doesn't change for `n=patience` epochs

```

#num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
#class GCN(torch.nn.Module):
#    def __init__(self):
#        super().__init__()
#        torch.manual_seed(1234)
#        self.conv1 = gnn.GCNConv(num_features, 16)
#        self.conv2 = gnn.GCNConv(16, 16)
#        self.conv3 = gnn.GCNConv(16, num_predictions)
#
#    def forward(self, x, edge_index, edge_weight):
#        x = self.conv1(x, edge_index, edge_weight)
#        x = x.relu()
#        #x = F.dropout(x, p=0.5, training=self.training)
#        x = self.conv2(x, edge_index, edge_weight)
#        x = x.relu()
#        #x = F.dropout(x, p=0.5, training=self.training)
#        x = self.conv3(x, edge_index, edge_weight)
#        return x
#

```

```

#model = GCN()
#pos_weight = torch.tensor([20/1])
#criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
#y_post_processing = False
num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
edge_dim = dataset[0].edge_attr.shape[1]
class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        torch.manual_seed(1234)
        # self.conv1 = gnn.GINEConv(num_features, 16)
        # self.conv2 = gnn.GINEConv(16, 16)
        # self.conv3 = gnn.GINEConv(16, num_predictions)
        self.conv1 = gnn.GINEConv(nn.Sequential(nn.Linear(num_features, 16), nn.ReLU(), nn
            .Linear(16, 16)), edge_dim=edge_dim)
        self.conv2 = gnn.GINEConv(nn.Sequential(nn.Linear(16, 16), nn.ReLU(), nn.Linear
            (16, 16)), edge_dim=edge_dim)
        self.conv3 = gnn.GINEConv(nn.Sequential(nn.Linear(16, 16), nn.ReLU(), nn.Linear
            (16, num_predictions)), edge_dim=edge_dim)

    def forward(self, x, edge_index, edge_weight):
        x = self.conv1(x, edge_index, edge_weight)
        x = x.relu()
        x = self.conv2(x, edge_index, edge_weight)
        x = x.relu()
        x = self.conv3(x, edge_index, edge_weight)
        return x

model = GCN()
pos_weight = torch.tensor([20/1])
criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
y_post_processing = False
#num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
#class softGCN(torch.nn.Module):
#    def __init__(self):
#        super().__init__()
#        torch.manual_seed(1234)
#        self.conv1 = gnn.GCNConv(num_features, 16)
#        self.conv2 = gnn.GCNConv(16, 16)
#        self.conv3 = gnn.GCNConv(16, num_predictions)
#
#    def forward(self, x, edge_index, edge_weight):
#        x = self.conv1(x, edge_index, edge_weight)
#        x = x.relu()
#        x = self.conv2(x, edge_index, edge_weight)
#        x = x.relu()
#        x = self.conv3(x, edge_index, edge_weight)
#        # x = gnn_utils.softmax(x, dim=0)
#        return x
#
#model = softGCN()
#pos_weight = torch.tensor([20/1])
#criterion = torch.nn.CrossEntropyLoss()
#y_post_processing = True

```

```

#num_features, num_predictions = dataset[0].x.shape[1], dataset[0].y.shape[1]
#class GAT(torch.nn.Module):
#    def __init__(self):
#        super().__init__()
#        torch.manual_seed(1234)
#        self.conv1 = gnn.GATConv(num_features, 16)
#        self.conv2 = gnn.GATConv(16, 16)
#        self.conv3 = gnn.GATConv(16, num_predictions)
#
#    def forward(self, x, edge_index, edge_weight):
#        x = self.conv1(x, edge_index, edge_weight)
#        x = x.tanh()
#        x = self.conv2(x, edge_index, edge_weight)
#        x = x.tanh()
#        x = self.conv3(x, edge_index, edge_weight)
#        return x
#
#model = GAT()
#pos_weight = torch.tensor([20/1])
#criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
#y_post_processing = False
#
# pos_weight = torch.tensor([len(y)/sum(y)])
# pos_weight = torch.tensor([20/1])
# criterion = torch.nn.BCELoss()
learning_rate=0.01
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5)

```

5.3 Training Routine

optimizer and criterion sourced from previous block. This is our test iteration

- `.zero_grad()` is a memory optimization while training by setting gradients to `None` type
- `.backwards()` computes gradient given loss (vanishing gradients occur here)
- `.step()` updates weights from the gradient calculation
- `y_post_processing` is a global var

```

def train(model, data):
    optimizer.zero_grad()
    # print(data.x)
    # print(data.y)
    # print(data.edge_attr)
    # print(data)
    out = model(
        data.x, data.edge_index, data.edge_attr.squeeze()
        # data.x, data.edge_index, data.edge_attr.squeeze(-2)
    )
    if y_post_processing:
        loss = criterion(out.squeeze(-1), data.y.squeeze(-1).argmax())
    else:

```

```

        loss = criterion(out, data.y)
    loss.backward()
    optimizer.step()
    return loss

```

5.4 Testing Function

For a circular IID graph, a perfect model should predict $p(v) = \pm 1/2$ if $v_i \pmod{n}/2 \equiv v_j \pmod{n}/2$ since that is the absolute most certain a model can get under the case of symmetric observers.

This is our testing loop:

- We evaluate via a linear classifier where >0.5 is treated as a guess of source (may reevaluate how loss is actually calculated)

```

def test(model, dataset):
    all_preds = []
    all_truths = []
    for data in dataset:
        preds = model(
            data.x, data.edge_index, data.edge_attr.squeeze()
        )
        print(f"Data:_{data.x[:, :1].flatten()}")
        if y_post_processing:
            preds = torch.softmax(preds, dim=0)
            print(f"Prob:_{preds.flatten().data}")
        else:
            preds = torch.sigmoid(preds)
            print(f"Conf:_{preds.flatten().data}")
        if y_post_processing:
            pred = preds.argmax().squeeze()
            preds = torch.zeros(preds.shape)
            preds[pred] = 1
        else:
            preds = (preds > 0.5).float()
            print(f"True:_{data.y.flatten()}")
            print(f"Pred:_{preds.flatten().data}")
            all_preds.append(preds.squeeze())
            all_truths.append(data.y.flatten())
    preds = torch.cat(all_preds)
    truths = torch.cat(all_truths)

    tp = ((preds == 1) & (truths == 1)).sum().float()
    tn = ((preds == 0) & (truths == 0)).sum().float()
    fp = ((preds == 1) & (truths == 0)).sum().float()
    fn = ((preds == 0) & (truths == 1)).sum().float()

    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp)
    recall = tp / (tp + fn)
    f1 = 2 * precision * recall / (precision + recall)
    return accuracy, precision, recall, f1

```

5.5 Model Routine

Final Training Cycle:

```
model.train()
for epoch in range(1000):
    epoch_loss = 0
    print("Individual Losses:", end="")
    for data in train_data:
        loss = train(model, data)
        epoch_loss = epoch_loss + loss
        print(f"{loss.item():.4f}", end=", ")
    print("")
    print(f'Epoch: {epoch:03d}, Loss: {epoch_loss:.4f}')
```

Testing Eval:

```
model.eval()
err = test(model, test_data)
print(f'Accuracy: {err[0]:.4f}, Precision: {err[1]:.4f}, Recall: {err[2]:.4f}, F1: {err[3]:.4f}')
```